

Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

Wydział Fizyki i Informatyki Stosowanej

Interfejsy użytkownika i biblioteki graficzne

Eksplorator fraktali

Zarys projektu do akceptacji

Autorzy

Jakub Piela (kapitan)
Marcel Gładysz
Patryk Pilch

Prowadzący: dr inż. Janusz Malinowski
Temat: własny
Rok akademicki: 2026/2027
Data złożenia: 02.11.2026

1 Opis projektu

Eksplorator fraktali to program w C++ do generowania, interaktywnego oglądania i animowania fraktali z trzech rodzin:

- **fraktale zespolone** – zbiór Mandelbrota i zbiory Julii,
- **fraktale IFS** – atraktory układów przekształceń afinicznych, np. trójkąt Sierpińskiego i paproć Barnsleya,
- **L-systemy** – gramatyki przepisywania napisów rysowane grafiką żółwia, np. krzywa Kocha i rośliny.

Użytkownik przybliża i przesuwa widok myszą, zmienia parametry i palety kolorów w panelu sterowania, a bieżący obraz zapisuje do pliku. Program tworzy też **animację płynnego przejścia („morfingu") między fraktalami IFS**: przeliczając współczynniki przekształceń z jednego fraktala na drugi, generuje serię bitmap zapisywanych na dysku.

Każdy piksel obrazu liczy nasz kod na procesorze. Biblioteki służą wyłącznie do otwarcia okna, obsługi zdarzeń, wyświetlenia gotowego bufora pikseli i narysowania panelu sterowania.

Animacja przejścia, plik scenariusza i fraktale trójwymiarowe nawiązują do niewybranego tematu nr 25 („Generowanie i animacja fraktali"). Nasz projekt obejmuje jego zakres dotyczący fraktali IFS i rozszerza go o fraktale zespolone oraz L-systemy.

2 Założenia wstępne

- **Język i środowisko:** C++17, Visual Studio 2026, Windows 11, kompilacja x64. SFML jest linkowany statycznie, więc program działa bez dodatkowych plików DLL.
- **Biblioteki z wykładu:** SFML 3.1 (okno, zdarzenia, wyświetlenie bufora pikseli), Dear ImGui z nakładką ImGui-SFML (panel sterowania) oraz ImGuiFileDialog (okna wyboru plików).
- **Bez OpenGL i DirectX w naszym kodzie:** obliczenia, kolorowanie, rysowanie linii i rzutowanie fraktali 3D wykonuje nasz kod na CPU.
- **Działanie bez plików wejściowych:** wszystkie parametry można ustawić w panelu. Pliki definicji i scenariusza są opcjonalne i służą do zapisu oraz wczytywania gotowych ustawień.
- **Płynność:** długie obliczenia (animacja, duże obrazy) działają w osobnym wątku. Interfejs nie zamarza, widać pasek postępu i można przerwać generowanie.
- **Okno** jest skalowalne. Po zmianie rozmiaru obraz jest przeliczany w nowej rozdzielczości, a nie rozciągany.
- **Oddanie projektu:** archiwum `numer_projektu.7z` (kompresja Ultra) z katalogami `DOC`, `SRC` i `BIN`; dokumentacja w PDF zgodna ze wzorcem.

3 Wymagania

3.1 Wymagania podstawowe

1. **Zbiór Mandelbrota:** przybliżanie kółkiem myszy w punkt pod kursorem, przesuwanie przeciąganiem, powrót do widoku startowego, regulowana maksymalna liczba iteracji.
2. **Zbiory Julii:** parametr c wybierany kliknięciem na zbiorze Mandelbrota albo wpisywany w panelu.

3. **Kolorowanie zbiorów zespolonych:** płynne (bez widocznych pasów), co najmniej trzy palety do wyboru.
4. **Fraktale IFS:** rysowanie grą w chaos; wbudowane trójkąt Sierpińskiego, paproć Barnsleya i smok Heighwaya; edycja współczynników w panelu oraz wczytywanie i zapis definicji w pliku tekstowym. Kolor punktu zależy od użytego przekształcenia, a fraktal jest automatycznie skalowany tak, aby mieścił się w oknie i nie był za mały.
5. **Animacja przejścia IFS:** płynne przejście między co najmniej dwoma fraktalami w zadanej liczbie klatek, podgląd klatek w trakcie generowania, zapis ponumerowanej serii obrazów PNG lub BMP. Parametry ustawiane w panelu albo wczytywane z pliku scenariusza.
6. **L-systemy:** aksjomat, reguły, kąt i liczba iteracji z panelu lub pliku; rysowanie grafiką żółwia ze stosem rozgałęzień; wbudowane krzywa Kocha, smok Heighwaya i roślina; automatyczne dopasowanie rysunku do okna.
7. **Zapis obrazu:** bieżący widok do pliku PNG lub BMP.
8. **Wydajność:** zbiory zespolone liczone wielowątkowo, tak aby nawigacja była płynna.
9. **Okno:** poprawne działanie przy zmianie rozmiaru, bez rozciągania obrazu.

3.2 Wymagania poszerzone

- **Fraktale IFS 3D:** przekształcenia przestrzenne (12 współczynników), własne rzutowanie perspektywiczne, pozycja obserwatora ustawiana w panelu i obrót kamery myszą. Punkty dalsze są ciemniejsze i nie przesłaniają bliższych (własny bufor głębi).
- **Animacja przejścia dla fraktali 3D.**
- **Animacja przybliżenia** zbiorów Mandelbrota i Julii eksportowana jako seria klatek.
- **Edytor palet:** gradient definiowany punktami kontrolnymi, zapis i wczytywanie palet.
- **Kolejne fraktale zespolone:** Burning Ship, Multibrot (z^n), fraktal Newtona dla wielomianu $z^3 - 1$.
- **Zakładki widoku:** zapis i odtwarzanie środka, skali, palety i liczby iteracji.
- **Zapis w wysokiej rozdzielczości** z antyaliasingiem (supersampling).
- **Głębokie przybliżenie:** arytmetyka o zwiększonej precyzji, gdy `double` przestaje wystarczać.

4 Analiza projektu

4.1 Specyfikacja danych wejściowych

Rodzaj	Format	Zawartość
Interakcja	mysz, klawiatura, panel	przybliżanie, przesuwanie, wybór c, parametry fraktali, paleta
Definicja IFS	plik `.ifs`	jedno przekształcenie w wierszu: `a b c d e f [p]`
Scenariusz animacji	plik `.anim`	rozmiar klatek, liczba punktów, fraktale kluczowe, liczba klatek przejść
L-system	plik `.lsys`	aksjomat, reguły w postaci `F=F+F-F`, kąt, liczba iteracji

Przekształcenie IFS ma postać $x' = ax + by + e$, $y' = cx + dy + f$. Prawdopodobieństwo p jest opcjonalne. Gdy go brak, program wyznacza je z wyznaczników przekształceń (punkt 5.2). Wiersze zaczynające się od `#` są komentarzami. Przykładowy scenariusz przejścia z trójkąta Sierpińskiego w paproć Barnsleya w 100 klatkach:

```
# szerokosc wysokosc punkty_na_klatke
800 600 2000000
# liczba fraktali kluczowych
2
# fraktal 1: liczba przeksztalcen, potem wiersze a b c d e f [p]
3
0.5 0 0 0.5 0    0
0.5 0 0 0.5 0.5  0
0.5 0 0 0.5 0.25 0.5
# liczba klatek przejscia do fraktala 2
100
# fraktal 2
4
 0.00  0.00  0.00  0.16  0  0     0.01
 0.85  0.04 -0.04  0.85  0  1.6   0.85
 0.20 -0.26  0.23  0.22  0  1.6   0.07
-0.15  0.28  0.26  0.24  0  0.44  0.07
```

Plik L-systemu:

```
axiom F
rule F=F+F-F-F+F
angle 90
iterations 4
```

Błędne pliki (zła liczba wartości, niepoprawne liczby, brak sekcji) nie zamykają programu. Wyświetlamy komunikat z numerem wiersza, w którym wystąpił błąd.

4.2 Opis oczekiwanych danych wyjściowych

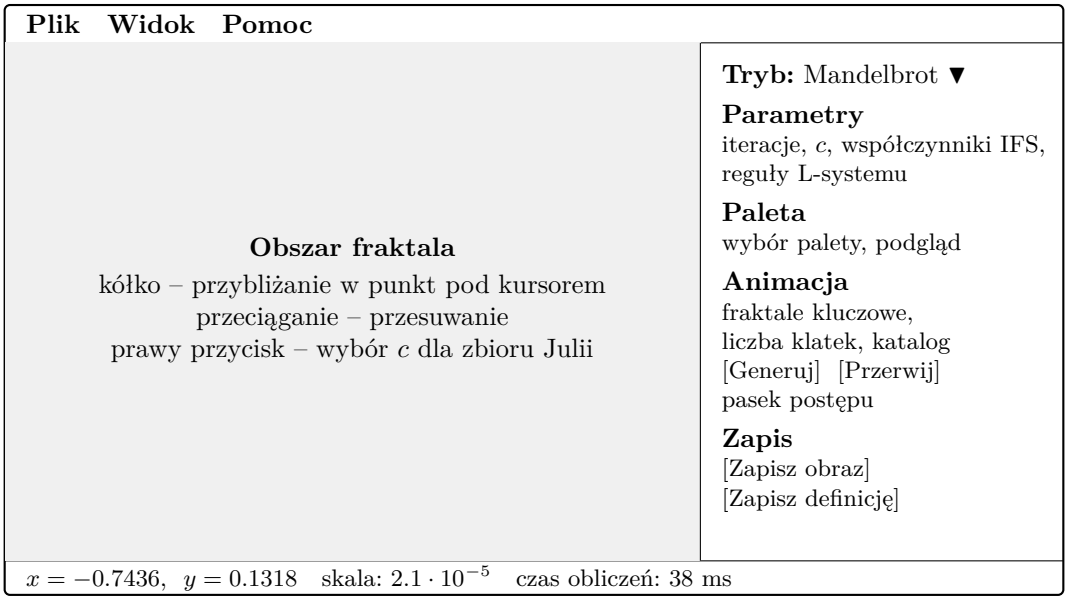
- **Obraz:** bieżący widok zapisany jako PNG lub BMP w rozmiarze okna albo w zadanej rozdzielczości.
- **Animacja:** seria ponumerowanych plików `klatka_0000.png`, `klatka_0001.png`, … w wybranym katalogu. Kolejno oglądane tworzą płynną animację.
- **Definicje:** bieżące ustawienia IFS, L-systemu lub scenariusza zapisane do pliku w formatach opisanych powyżej.

4.3 Struktury danych

Struktura	Zawartość
`Image`	bufor pikseli `std::vector<std::uint32_t>` (RGBA) oraz szerokość i wysokość
`ComplexView`	środek widoku (c_x, c_y), skala (jednostki na piksel), maksymalna liczba iteracji
`Palette`	lista punktów kontrolnych gradientu (pozycja t, kolor)
`AffineMap`	współczynniki a, b, c, d, e, f i prawdopodobieństwo p
`Ifs`	nazwa i `std::vector<AffineMap>`
`AffineMap3D`	macierz 3×3, wektor przesunięcia i p (poszerzenie)
`AnimationScenario`	rozmiar klatek, liczba punktów na klatkę, fraktale kluczowe, liczby klatek przejść
`LSystem`	aksjomat, reguły `std::map<char, std::string>`, kąt, liczba iteracji
`TurtleState`	pozycja (x, y) i kierunek żółwia; rozgałęzienia na `std::stack<TurtleState>`

Każdy rodzaj fraktala implementuje wspólną klasę bazową `FractalRenderer` z metodą `render(Image&)`. Klasy pochodne to `MandelbrotRenderer`, `JuliaRenderer`, `IfsRenderer` i `LSystemRenderer`. Dzięki temu zmiana trybu w interfejsie sprowadza się do wyboru innego obiektu.

4.4 Specyfikacja interfejsu użytkownika



Okno składa się z menu (wczytywanie i zapis plików, powrót do widoku startowego, informacje o programie), obszaru fraktala, panelu sterowania po prawej i paska stanu. Pasek stanu pokazuje współrzędne punktu pod kursorem, bieżącą skalę i czas obliczenia klatki. Elementy, których nie da się w danej chwili użyć, są wyszarzone, np. zapis klatek przed wygenerowaniem animacji. Panel i okna dialogowe tworzymy w Dear ImGui.

4.5 Wyodrębnienie i zdefiniowanie zadań

1. **Rdzeń aplikacji i interfejs:** okno, pętla zdarzeń, panel ImGui, nawigacja myszą, okna dialogowe, zmiana rozmiaru okna, integracja modułów.
2. **Fraktale zespolone:** Mandelbrot i Julia, płynne kolorowanie, palety, obliczenia wielowątkowe.

3. **IFS i animacja:** parser plików, gra w chaos, kolorowanie, dopasowanie do okna, przejścia między fraktalami, eksport klatek w wątku roboczym.
4. **L-systemy:** parser, przepisywanie napisów, grafika żółwia, rysowanie odcinków w buforze pikseli.
5. **Część wspólna:** klasa `Image`, zapis PNG i BMP, obsługa błędów wczytywania.

4.6 Decyzja o wyborze narzędzi programistycznych

- **Visual Studio 2026 i C++17** – znamy je z zajęć. Standard daje `std::thread` i `std::atomic` do wątków, `<random>` do losowania w grze w chaos oraz `<filesystem>` do zapisu klatek.
- **SFML 3.1** – poznana na wykładzie i laboratoriach, prosta obsługa okna, zdarzeń i wyświetlania bufora przez `sf::Texture`. `sf::Image` zapisuje obrazy PNG i BMP.
- **Dear ImGui, ImGui-SFML, ImGuiFileDialog** – omawiane na wykładzie. Pozwalają szybko zbudować panel z suwakami, polami liczbowymi i oknami wyboru plików.
- **Git** (wspólne repozytorium) do pracy zespołowej, LaTeX do dokumentacji, 7-Zip do archiwum.

5 Algorytmy

5.1 Zbiory Mandelbrota i Julii

Dla każdego punktu płaszczyzny zespolonej powtarzamy przekształcenie

$$z_{n+1} = z_n^2 + c.$$

W zbiorze Mandelbrota $z_0 = 0$, a c jest współrzędną piksela. W zbiorze Julii c jest stałe dla całego obrazu, a z_0 jest współrzędną piksela. Gdy $|z_n|^2 > 4$, ciąg ucieka do nieskończoności i punkt nie należy do zbioru. Porównujemy kwadrat modułu, więc nie liczymy pierwiastka.

Piksel i płaszczyzna zespolona. Piksel (p_x, p_y) odpowiada punktowi $x = c_x + (p_x - W/2)\,s$, $y = c_y - (p_y - H/2)\,s$, gdzie s to skala. Przy przybliżaniu o czynnik k nowy środek wynosi $\mathbf{m}' = \mathbf{q} + (\mathbf{m} - \mathbf{q})\,k$, gdzie $\mathbf{q}$ to punkt pod kursorem. Dzięki temu punkt pod kursorem zostaje w miejscu.

Płynne kolorowanie. Zamiast całkowitej liczby kroków n używamy wartości ułamkowej

$$\nu = n + 1 - \log_2(\ln |z_n|),$$

którą przeliczamy na kolor z palety. Usuwa to pasy kolorów. Punkty należące do zbioru są czarne.

5.2 Fraktale IFS – gra w chaos

Zaczynamy od dowolnego punktu. W każdym kroku losujemy przekształcenie w_i z prawdopodobieństwem p_i, stosujemy je do punktu i zaznaczamy wynik. Pierwsze 20 punktów pomijamy, bo nie leżą jeszcze na atraktorze. Gdy prawdopodobieństw nie podano, przyjmujemy je proporcjonalne do pola, jakie przekształcenie zachowuje:

$$p_i = \frac{\max(|a_i d_i - b_i c_i|,\ \varepsilon)}{\sum_j \max(|a_j d_j - b_j c_j|,\ \varepsilon)}.$$

Dzięki temu punkty rozkładają się na fraktalu równomiernie. Minimalna wartość ε chroni przekształcenia spłaszczające, np. łodygę paproci.

Dopasowanie do okna. Najpierw liczymy kilka tysięcy punktów, aby wyznaczyć prostokąt otaczający atraktor. Dodajemy margines 5% i dobieramy skalę z zachowaniem proporcji, tak aby fraktal wypełniał okno.

Kolorowanie. Każde przekształcenie ma własny kolor, a punkt przyjmuje kolor przekształcenia, które go wyznaczyło. Opcjonalnie zliczamy trafienia w piksel i ustalamy jasność logarytmicznie: jasność $= \ln(1+h)/\ln(1+h_{\max})$.

5.3 Animacja przejścia między fraktalami IFS

Niewielka zmiana współczynników przekształceń daje niewielką zmianę obrazu. Dlatego klatkę k z K w przejściu od fraktala A do fraktala B tworzymy, interpolując każdy współczynnik:

$$w_i^{(k)} = (1-s)\, w_i^A + s\, w_i^B, \qquad s = t^2(3-2t), \quad t = k/K.$$

Funkcja $s(t)$ łagodzi początek i koniec przejścia.

- **Różna liczba przekształceń.** Do zbioru z mniejszą liczbą przekształceń dokładamy kopie istniejących, dzieląc prawdopodobieństwo kopiowanego przekształcenia na pół. Dwie identyczne kopie dają ten sam atraktor, więc pierwsza klatka się nie zmienia.
- **Poprawność klatek pośrednich.** Jeśli przekształcenia A i B są zwężające, to ich kombinacja wypukła też jest zwężająca: $\|(1-s)M_A + sM_B\| \le (1-s)\|M_A\| + s\|M_B\| < 1$. Każda klatka jest więc poprawnym fraktalem.
- **Stabilny kadr.** Prostokąt otaczający liczymy dla każdej klatki i wygładzamy go średnią kroczącą, aby obraz nie „pływał”.
- **Eksport.** Klatki liczy wątek roboczy. Każdą gotową klatkę zapisuje do pliku i przekazuje do podglądu. Flaga `std::atomic<bool>` pozwala przerwać generowanie.

5.4 L-systemy i grafika żółwia

L-system składa się z aksjomatu, reguł przepisywania i kąta. W każdej iteracji wszystkie symbole napisu zastępujemy jednocześnie według reguł. Długość napisu rośnie wykładniczo, więc ograniczamy ją, np. do 5 mln symboli, i ostrzegamy użytkownika.

Symbol	Polecenie dla żółwia
`F`	idź naprzód o krok i rysuj odcinek
`f`	idź naprzód o krok bez rysowania
`+`, `-`	obróć się w lewo lub w prawo o zadany kąt
`[`	zapamiętaj pozycję i kierunek (odłóż na stos)
`]`	wróć do ostatnio zapamiętanego stanu (zdejmij ze stosu)

Żółw przechodzi napis dwa razy. Za pierwszym razem tylko wyznacza prostokąt otaczający rysunek, aby dobrać skalę. Za drugim rysuje odcinki algorytmem Bresenhama bezpośrednio w buforze pikseli.

5.5 Fraktale IFS 3D (poszerzenie)

Przekształcenie przestrzenne ma 12 współczynników: $\mathbf{x}' = M\mathbf{x} + \mathbf{t}$, gdzie M to macierz 3×3. Grę w chaos prowadzimy w przestrzeni. Każdy punkt przekształcamy do układu kamery i rzutujemy perspektywicznie: $x_s = f\, x_c / z_c + W/2$. Własny bufor głębi przechowuje dla piksela odległość najbliższego punktu, więc dalsze punkty nie przesłaniają bliższych. Jasność maleje z odległością od obserwatora.

5.6 Obliczenia wielowątkowe

Obraz dzielimy na poziome pasy, po jednym na wątek (`std::thread::hardware_concurrency()`). Każdy wątek zapisuje tylko swoje wiersze, więc nie potrzeba blokad. W grze w chaos każdy wątek ma własny generator liczb losowych i własny bufor trafień, które na końcu sumujemy. Podczas przeciągania liczymy podgląd w niższej rozdzielczości, a pełny obraz po puszczeniu przycisku myszy.

6 Podział pracy i analiza czasowa

Dokumentację i testy prowadzimy wspólnie. Tygodnie liczymy od akceptacji tematu.

Etap	Jakub Piela interfejs, zbiory zespolone	Marcel Gładysz IFS i animacja	Patryk Pilch L-systemy
Tydz. 1–2 szkielet	okno, bufor pikseli, panel ImGui, nawigacja myszą, odwzorowanie piksel–płaszczyzna, Mandelbrot	parser `.ifs`, gra w chaos	parser `.lsys`, przepisywanie napisów
Tydz. 3–4 rdzeń	Julia (wybór c kliknięciem), płynne kolorowanie, palety	wbudowane fraktale (Sierpiński, paproć), edycja współczynników, autoskalowanie, kolorowanie	grafika żółwia z rozgałęzieniami, wbudowane krzywe, dopasowanie do okna
Tydz. 5–6 komplet podstaw	okna dialogowe, zapis obrazu PNG/BMP, integracja modułów	animacja przejścia, eksport ponumerowanych klatek	wczytywanie i zapis definicji, okna dialogowe
Tydz. 7–8 poszerzenia	wielowątkowość i animacja w tle z paskiem postępu, animacja zoomu, zapis w wysokiej rozdzielczości, testy, archiwum 7z	plik scenariusza, fraktale IFS 3D	kolejne zbiory zespolone (Burning Ship, Multibrot, Newton), edytor palet

Wymagania podstawowe mają być gotowe po szóstym tygodniu. Ostatnie dwa tygodnie zostają na poszerzenia, testy i dokumentację w PDF.

7 Plan testów

- **Testy modułów:** parsery na poprawnych i celowo błędnych plikach (oczekiwany komunikat z numerem wiersza); znane wyniki: punkt $c = 0$ należy do zbioru Mandelbrota, a $c = 1$ ucieka po 3 krokach; liczba symboli `F` w krzywej Kocha po n iteracjach wynosi 5^n.
- **Testy powiązań:** zmiana trybu w trakcie obliczeń, zapis obrazu po wczytaniu definicji, przerwanie eksportu animacji.
- **Testy całościowe:** scenariusz przejścia trójkąta Sierpińskiego w paproć w 100 klatkach (przykład powyżej), zmiana rozmiaru okna, pomiar czasu obliczeń klatki.
- **Niezmienniki:** liczba zapisanych klatek równa sumie klatek przejść; obraz zbioru zespolonego nie zależy od liczby wątków.

8 Ryzyka i plan awaryjny

- **Wydajność przy dużym przybliżeniu** – podgląd w niższej rozdzielczości podczas przesuwania, pełny obraz po zatrzymaniu.
- **Precyzja `double`** – wymagania podstawowe ograniczamy do przybliżeń, przy których `double` wystarcza; głębokie przybliżenie jest poszerzeniem.
- **Bardzo długie napisy L-systemów** – limit długości i ostrzeżenie zamiast zawieszenia programu.
- **Zakres** – jeśli zabraknie czasu, wymagania podstawowe pozostają nietknięte, a odpadają pozycje z listy poszerzeń, od końca.